

Esterian Conquest

Sysop Manual

Mason A. Green

Draft — March 2026

Contents

1. Introduction	3
1.1. Terminology	3
2. Installation and Requirements	3
2.1. System Requirements	3
2.2. Building from Source	3
2.3. Directory Layout	3
3. Game Setup	4
3.1. Initializing a New Game	4
3.2. The setup.kdl File	4
3.2.1. game Node	4
3.2.2. setup_options Node	4
3.2.3. maintenance_days Node	5
4. Game Directory Structure	5
4.1. Core Files	5
4.2. Subdirectories	5
5. Configuration	5
5.1. config.kdl	5
5.1.1. Full Example	5
5.1.2. Top-Level Fields	6
5.1.3. session Block	6
5.1.4. inactivity Block	6
5.2. Environment Variables	7
6. Theming	7
6.1. Theme File Location	7
6.2. theme.kdl Format	7
6.3. Color Formats	8
6.4. Color Mode	8
6.5. Semantic Style Tokens	8
6.6. ANSI Off Mode	9
7. BBS Door Setup	10
7.1. Flags for Door Mode	10
7.2. Drop File	10
7.3. Enigma BBS (Rust Client)	10
8. SSH Access	10
9. Local and Direct Play	11
10. Turn Processing and Maintenance	11
11. Player Management	11
12. Classic Compatibility	11
13. CLI Reference	11
13.1. ec-cli	11
13.2. ec-client	11
14. Theme Token Reference	12

1. Introduction

Esterian Conquest (EC) is a multi-player strategy game originally written for DOS-era BBS systems. The Rust-native stack reimplements the game engine, player client, and admin tooling for modern systems while preserving classic compatibility at a well-defined import/export boundary.

This manual is for the **sysop** — the person responsible for hosting a game instance. It covers:

- installing and initializing a game
- configuration and theming
- BBS door, SSH, and local deployment
- player management
- turn processing and maintenance

For player-facing rules and gameplay, see the **Player Manual**.

1.1. Terminology

game directory The directory containing `ecgame.db` and related files for a single running game instance. Passed to all tools via `--dir`.

ec-cli The Rust command-line admin tool for setup, maintenance, and compatibility operations.

ec-client The Rust TUI player client.

ecgame.db The SQLite database that is the runtime source of truth for the Rust engine.

setup.kdl The KDL configuration file used to initialize a new game via `ec-cli sysop new-game`.

theme.kdl The KDL file that controls the visual style of `ec-client` for all players in this game directory.

config.kdl The sysop-managed runtime configuration file in the game directory. Bootstrapped from the bundled default on first run. Controls snoop mode, session timeouts, inactivity thresholds, game name, and theme path. Changes take effect on the next `ec-client` startup.

2. Installation and Requirements

2.1. System Requirements

- Linux, macOS, or Windows
- Rust toolchain (stable) to build from source, or a pre-built binary release
- A terminal emulator for local/SSH play; a BBS server with socket support for door deployment

2.2. Building from Source

From the repository root:

```
cargo build --release -p ec-cli -p ec-client
```

The release binaries will be in `target/release/`.

2.3. Directory Layout

A running game occupies a single directory. A minimal populated game directory looks like:

```

/path/to/mygame/
  ecgame.db          runtime database (SQLite)
  config.kdl         sysop runtime config (bootstrapped on first run)
  theme.kdl          visual theme (bootstrapped on first run)
  exports/           default export root for classic .DAT output
  queue/             default turn order queue directory

```

All tools take `--dir /path/to/mygame` to locate the game.

3. Game Setup

3.1. Initializing a New Game

Use `ec-cli sysop new-game` with a `setup.kdl` config file:

```
ec-cli sysop new-game --dir /path/to/mygame --config setup.kdl
```

This creates `ecgame.db` in the target directory, materialized from the declarative setup configuration.

3.2. The `setup.kdl` File

`setup.kdl` is a KDL document that declares the initial game parameters. It is consumed once at game creation; subsequent changes require a new game or direct database edits.

A minimal example:

```

game player_count=4 seed=1515

setup_options snoop=#true local_timeout=#false remote_timeout=#true \
  max_key_gap_minutes=10 minimum_time_minutes=0 \
  purge_after_turns=0 autopilot_after_turns=0

maintenance_days {
  day "sun"
  day "mon"
  day "tue"
  day "wed"
  day "thu"
  day "fri"
  day "sat"
}

```

3.2.1. `game` Node

Field	Type	Description
<code>player_count</code>	integer	Number of empires. Supported range: 1–25.
<code>seed</code>	integer	Optional. Map generation seed for reproducible layouts.

3.2.2. `setup_options` Node

These values seed the initial runtime settings when the game is first created. After creation, these settings are owned by `config.kdl` in the game directory (see Section 5). To change them on a running game, edit `config.kdl` directly — not `setup.kdl`.

Field	Type	Description
snoop	bool	Initial snoop mode. Overridable at runtime via <code>config.kdl</code> .
local_timeout	bool	Initial local session timeout setting.
remote_timeout	bool	Initial remote session timeout setting.
max_key_gap_minutes	integer	Initial maximum idle time before timeout.
minimum_time_minutes	integer	Initial minimum session time guarantee.
purge_after_turns	integer	Initial inactivity purge threshold. 0 = disabled.
autopilot_after_turns	integer	Initial autopilot threshold. 0 = disabled.

3.2.3. maintenance_days Node

Specifies which days of the week maintenance runs. Each child day node takes one of: "sun", "mon", "tue", "wed", "thu", "fri", "sat".

4. Game Directory Structure

4.1. Core Files

ecgame.db The SQLite runtime database. All game state lives here. Do not edit manually; use `ec-cli` for admin operations.

theme.kdl Visual theme for `ec-client`. Bootstrapped from the bundled default on first run if absent. Sysop-owned once created; not silently overwritten. See Section 6.

config.kdl Sysop runtime configuration. Bootstrapped from the bundled default on first run. Edit to change snoop mode, session timeouts, inactivity thresholds, game name, or theme path. See Section 5.

4.2. Subdirectories

exports/ Default root for classic .DAT export output. Can be overridden with `--export-root` or the `EC_CLIENT_EXPORT_ROOT` environment variable.

queue/ Default directory for turn order queue files. Can be overridden with `--queue-dir` or the `EC_CLIENT_QUEUE_DIR` environment variable.

5. Configuration

5.1. config.kdl

`config.kdl` is the sysop runtime configuration file. It lives in the game directory alongside `ecgame.db`. If absent when `ec-client` starts, it is bootstrapped from the bundled default automatically.

Changes to `config.kdl` take effect on the next `ec-client` startup. The settings are applied into the runtime database at that point; no manual database edits are required.

5.1.1. Full Example

```
// Display name shown in the main menu header.
game_name "Esterian Conquest"
```

```
// Theme file (relative to game directory, or absolute path).
// Remove this line to use theme.kdl in the same directory.
// theme "theme.kdl"

// Sysop snoop: set to #false to disable.
snoop #true

// Session timeout and timing policies.
session {
    // Minutes of inactivity before timeout (0-120).
    max_idle_minutes 10
    // Minimum time granted per session in minutes (0-120).
    minimum_time_minutes 0
    // Apply timeout to local (non-remote) sessions.
    local_timeout #false
    // Apply timeout to remote sessions.
    remote_timeout #true
}

// Inactivity thresholds (in turns). Set to 0 to disable.
inactivity {
    // Purge a player after this many inactive turns (0-100).
    purge_after_turns 0
    // Put a player on autopilot after this many inactive turns (0-100).
    autopilot_after_turns 0
}
```

5.1.2. Top-Level Fields

Field	Type	Default	Description
game_name	string	"Esterian Conquest"	Display name shown in the main menu header.
theme	string	<i>(absent)</i>	Theme file path, relative to the game directory. Omit to use theme.kdl.
snoop	bool	#true	Enable sysop snoop mode.

5.1.3. session Block

Field	Type	Default	Description
max_idle_minutes	integer	10	Minutes of inactivity before session timeout. Range: 0-120.
minimum_time_minutes	integer	0	Minimum session time guarantee in minutes. Range: 0-120.
local_timeout	bool	#false	Apply timeout to local (non-remote) sessions.
remote_timeout	bool	#true	Apply timeout to remote sessions.

5.1.4. inactivity Block

Field	Type	Default	Description
-------	------	---------	-------------

purge_after_turns	integer	0	Turns of inactivity before a player is purged. 0 = disabled. Range: 0–100.
autopilot_after_turns	integer	0	Turns of inactivity before autopilot engages. 0 = disabled. Range: 0–100.

NOTE: `config.kdl` is for sysop use only. Players do not interact with it. Fields not present in the file use their default values. Omitting a field is equivalent to setting it to its default.

5.2. Environment Variables

Variable	Description
EC_CLIENT_EXPORT_ROOT	Override the default export root directory.
EC_CLIENT_QUEUE_DIR	Override the default turn queue directory.
COLORTERM	Color depth hint. <code>truecolor</code> or <code>24bit</code> enables 24-bit color.
TERM	Terminal type. A value containing <code>256color</code> enables 256-color mode.

6. Theming

`ec-client` uses a file-driven theme system. The theme defines the visual appearance for all players connecting to this game directory.

6.1. Theme File Location

`ec-client` resolves the theme in this order:

1. If `<game_dir>/config.kdl` contains a `theme` directive, use that path (relative to `game_dir`).
2. Otherwise, use `<game_dir>/theme.kdl`.
3. If `theme.kdl` does not exist, bootstrap it from the bundled default.

`config.kdl` itself is bootstrapped on first run if absent, so it is always present by the time this resolution runs. The `theme` directive within it is optional; omitting it falls through to step 2.

On parse error, the bundled default is used silently so a corrupted theme never prevents players from connecting.

6.2. theme.kdl Format

A theme file is a KDL document. Each visual element is declared as a `style` node with a name and child `fg`, `bg`, and optional `bold` fields.

```
style "body" {
    fg "white"
    bg "black"
}

style "logo" {
    fg "bright_blue"
    bg "black"
    bold #true
}
```

```

style "selected" {
    fg "black"
    bg "bright_blue"
}

```

The star decoration colors are declared separately:

```
star-colors "bright_blue" "bright_white" "white" "bright_yellow" "yellow" "bright_red"
```

6.3. Color Formats

Three color formats are supported:

Format	Example	Notes
Named ANSI-16	"bright_blue"	Safe for all terminals including BBS/door clients. Recommended default.
256-color index	"idx:208"	Requires <code>--color-mode 256</code> or <code>truecolor</code> . Downgraded to nearest named color in 16-color mode.
24-bit hex RGB	"#ff8800"	Requires <code>--color-mode truecolor</code> . Downgraded gracefully in lower color modes.

The bundled default theme uses named ANSI-16 colors only. Custom themes may use richer formats when targeting modern terminals.

6.4. Color Mode

`ec-client` selects a color mode at startup:

Mode	Description
<code>ansi16</code>	Classic 16-color ANSI. Safe for BBS/door and all terminal types.
<code>256</code>	256-color xterm palette.
<code>truecolor</code>	24-bit RGB. Best for modern local or SSH terminals.
<code>auto</code>	Detected from <code>COLORTERM</code> and <code>TERM</code> environment variables.

Override with the `--color-mode` flag:

```
ec-client --dir /path/to/mygame --player 1 --color-mode truecolor
```

When `--encoding cp437` is specified (BBS door mode), `ec-client` defaults to `ansi16` regardless of the environment, unless `--color-mode` is set explicitly.

6.5. Semantic Style Tokens

All required style tokens:

Token	Purpose
<code>body</code>	Default body text.
<code>title</code>	Screen and section titles.
<code>menu</code>	Menu item text.

menu_hotkey	Menu hotkey letters.
prompt	Input prompt text.
prompt_hotkey	Prompt hotkey letters.
prompt_notice_action	Action-notice accent in prompts.
bright	Emphasized body text.
logo	Title logo decoration.
intro_accent	Intro screen accent color.
intro_tribute	Intro screen tribute line.
stardate_label	Stardate label.
stardate_week	Stardate week value.
stardate_year	Stardate year value.
error	Error messages.
notice	Warning / notice messages.
status_label	Status bar labels.
status_value	Status bar values.
table_chrome	Table borders and separators.
table_header	Table column headers.
table_body	Table body rows.
disabled_row	Disabled / unavailable table rows.
selected	Selected / highlighted row.
alert	High-priority alert text.
help_header	Help overlay section headers.
help_panel	Help overlay body text.
map_dot	Star map background dots.
map_crosshair	Star map crosshair / cursor.
map_center	Star map center marker.
quote	Quote / flavor text body.
quote_author	Quote attribution.
report_header	Report page headers.
indicator_on	Active indicator (e.g. status lit).
indicator_off	Inactive indicator.

6.6. ANSI Off Mode

Players can toggle ANSI color off within a session (a session-level preference only — it does not modify `theme.kdl`). ANSI Off applies a monochrome projection over the loaded theme: white/black with preserved reverse-video selection. A new session always starts with ANSI On.

7. BBS Door Setup

`ec-client` can run as a BBS door. In door mode it reads from and writes to a socket instead of the local TTY, using CP437 encoding and 16-color ANSI.

7.1. Flags for Door Mode

```
ec-client \  
  --dir /path/to/mygame \  
  --player <1-based index> \  
  --encoding cp437 \  
  --color-mode ansi16
```

`--encoding cp437` switches box-drawing characters and other extended characters to the CP437 code page, which is required for correct rendering in classic ANSI-aware BBS terminals (SyncTERM, NetRunner, etc.).

When `--encoding cp437` is active, `--color-mode` defaults to `ansi16` automatically. Override only if you know your BBS clients support a richer color depth.

7.2. Drop File

The `--player` flag selects the 1-based empire index. Map this from your BBS drop file (e.g. the node's assigned player number) before invoking `ec-client`.

NOTE: The original DOS `ECGAME.EXE` v1.5 expects a strict 32-line WWIV-style `CHAIN.TXT` drop file. Enigma BBS-generated `DOOR.SYS` / `DORINFO` files will crash `ECGAME.EXE`. See `docs/sysop/enigma-bbs-setup.md` for the full legacy DOS door path if you need to host the original binary.

7.3. Enigma BBS (Rust Client)

To run the native `ec-client` as an Enigma BBS door, use the `abracadabra` module with `io: socket` and pass `--dir`, `--player`, `--encoding cp437`, and `--color-mode ansi16` as arguments. The client will inherit the socket from Enigma's stdio handoff.

8. SSH Access

`ec-client` runs cleanly over SSH. No special flags are required for SSH sessions on modern terminals.

Color mode is auto-detected from the environment:

- `COLORTERM=truecolor` → 24-bit RGB
- `TERM` containing `256color` → 256-color
- Otherwise → 16-color ANSI fallback

Force a specific mode with `--color-mode` if your SSH setup does not propagate `COLORTERM` reliably:

```
ec-client --dir /path/to/mygame --player 1 --color-mode 256
```

UTF-8 encoding (the default) is correct for SSH sessions on modern terminals. Use `--encoding cp437` only if you are proxying through a BBS or a CP437 terminal emulator over SSH.

9. Local and Direct Play

For localhost play, run `ec-client` directly in your terminal:

```
ec-client --dir /path/to/mygame --player 1
```

Color mode and encoding default to `auto` / `utf8`, which is correct for modern terminal emulators. The client detects `COLORTERM` and `TERM` automatically.

10. Turn Processing and Maintenance

This section will document the Rust maintenance cycle, `ec-cli maint`, turn submission, and scheduled processing. Placeholder pending implementation documentation.

11. Player Management

This section will document adding players, player record indices, purge and autopilot settings, and inactivity policies. Placeholder pending implementation documentation.

12. Classic Compatibility

This section will document the `.DAT` import/export boundary, oracle testing with the original binaries, and the preservation fixture workflow. Placeholder pending implementation documentation.

13. CLI Reference

13.1. `ec-cli`

```
ec-cli <subcommand> [options]
```

Key subcommand families:

Subcommand	Purpose
<code>sysop new-game</code>	Initialize a new game from a <code>setup.kdl</code> file.
<code>sysop</code>	Other admin and maintenance operations.
<code>compat export</code>	Export game state to classic <code>.DAT</code> format.
<code>compat import</code>	Import classic <code>.DAT</code> state into <code>ecgame.db</code> .
<code>report</code>	Generate turn reports.
<code>validate</code>	Validate game state against oracle fixtures.

13.2. `ec-client`

```
ec-client --dir <game_dir> --player <1-based index> [options]
```

Flag	Description
<code>--dir <path></code>	Game directory containing <code>ecgame.db</code> . Required.
<code>--player <N></code>	1-based empire index. Required.
<code>--encoding <utf8 cp437></code>	Output encoding. Default: <code>utf8</code> . Use <code>cp437</code> for BBS/door mode.

<code>--color-mode <ansi16 256>truecolor auto></code>	Color depth. Default: auto (env-detected). CP437 mode defaults to ansi16.
<code>--export-root <path></code>	Override export directory. Default: <code><game_dir>/exports</code> .
<code>--queue-dir <path></code>	Override turn queue directory. Default: <code><game_dir>/queue</code> .

14. Theme Token Reference

The default bundled theme values are listed below for reference. All colors use named ANSI-16 values.

Token	fg	bg	bold
body	white	black	—
title	bright_blue	black	yes
menu	white	black	—
menu_hotkey	yellow	black	yes
prompt	white	black	—
prompt_hotkey	yellow	black	yes
prompt_notice_action	bright_cyan	black	yes
bright	bright_white	black	yes
logo	bright_blue	black	yes
intro_accent	bright_blue	black	—
intro_tribute	bright_magenta	black	—
stardate_label	cyan	black	—
stardate_week	bright_cyan	black	—
stardate_year	yellow	black	—
error	red	black	yes
notice	bright_red	black	yes
status_label	white	black	—
status_value	white	black	—
table_chrome	blue	black	—
table_header	cyan	black	yes
table_body	bright_white	black	—
disabled_row	bright_black	black	—
selected	black	bright_blue	—
alert	bright_white	red	yes
help_header	bright_blue	black	yes
help_panel	white	black	—

map_dot	green	black	—
map_crosshair	bright_red	black	yes
map_center	bright_white	black	yes
quote	white	black	—
quote_author	white	black	—
report_header	cyan	black	—
indicator_on	bright_green	black	yes
indicator_off	bright_black	black	—

Star decoration colors (6 slots, cycling):

star-colors "bright_blue" "bright_white" "white" "bright_yellow" "yellow" "bright_red"